18 febbraio 2019



Politecnico di Milano

Anno accademico 2018-2019

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Margara	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

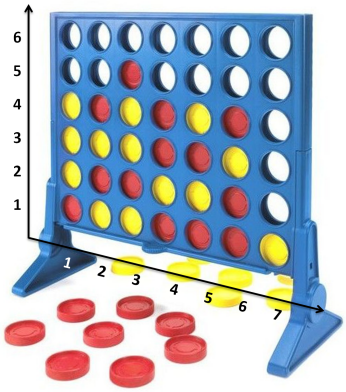
Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1



Si consideri il tipo di dato astratto `Connect4` che modella una partita al noto gioco "Forza 4" (vedi figura a lato).

Lo scopo del gioco é allineare quattro pedine dello stesso colore nella scacchiera di gioco sia in verticale, orizzontale o obliquo. A questo scopo due giocatori (uno dotato di 21 pedine gialle, l'altro di 21 pedine rosse) si alternano facendo scivolare la pedina nei binari prestabiliti dalla scacchiera.

Si supponga una dimensione della "griglia" come in figura (7 colonne per 6 righe). Una pedina viene depositata invocando il metodo `put` ed indicando la colonna in cui si vuole depositare la pedina (nell'intervallo `1-7`) e l'identificativo del giocatore. I due giocatori sono convenzionalmente identificati dai valori interi 1 e 2. Il metodo `get` ritorna 0 se la cella indicata è vuota, oppure 1 o 2 a seconda della pedina presente nella posizione indicata (del giocatore 1 o del giocatore 2). La cella in basso a sinistra ha posizione 1,1 (vedi figura).

```
public class Connect4 {
    // Crea una partita vuota.
    public Connect4() ;

    // Deposita la pedina del giocatore player (1 o 2) nella colonna indicata.
    // Restituisce l'eccezione FullColumnException se la colonna è piena.
    // Restituisce l'eccezione GameOverException se c'è già un vincitore.
    public void put(int column, int player) throws FullColumnException, GameOverException;

    // Restituisce 0 se la cella è libera, 1 o 2 se è occupata dalla pedina
    // del giocatore corrispondente. La posizione indicata deve appartenere
    // all'intervallo ammesso (1-7 x 1-6). La cella in basso a sinistra (vedi figura)
    // ha posizione 1,1.
    public /*@ pure @*/ int get(int column, int row) ;

    // Restituisce true sse c'è un vincitore (4 pedine dello stesso giocatore allineate).
    public /*@ pure @*/ boolean winner() ;

    // Svuota la griglia dando inizio a una nuova partita.
    public void clear() ;
}
```

Domanda a)

Si specifichino in JML i metodi `put`, `winner` e `clear`.

Soluzione

Per semplicitá (pur non essendo necessario) definiamo un'abbreviazione:

```
unchanged(row,column) =
(\forall int c; c>=1 && c<=7;
       (\forall int r; r>=1 && r<=6;
           get(c,r)==\old(get(c,r)) || (r==row && c==column))))
```

che afferma che ogni cella diversa da (row,column) non e' modificata. Una simile abbreviazione `unchanged` afferma che nessuna cella é modificata.

```
//@requires column>=1 && column<=7 and (player==1 || player==2)
//@ensures \old(get(column,6)==0) && \old(!winner()) &&
//@  (\exists int row; row>=1 && row<=6; \old(get(column,row)==0) && get(column,row)==player &&
//@     (\forall int r; r>=1 && r<row; \old(get(column,r)!=0)) &&
```

```
//@     unchanged(row,column)
//@signals(FullColumnException e) (\forall int r; r>=1 && r<=6; \old(get(column,r)!=0)) &&
//@                               unchanged;
//@signals(GameOverException e) \old(winner()) && unchanged;
public void put(int column, int player) throws FullColumnException, GameOverException;

//@ensures \result <=>
//@  (\exists int c; c>=1 && c<=7;
       (\exists int r; r>=1 && r<=6;
//@      (\exists int dx; dx>=-1 && dx<=1;
          (\exists(int dy; dy>=-1 && dy<=1;
//@         !(dx==0 && dy==0) && c+3*dx>=1 && c+3*dx<=7 && r+3*dy>=1 && r+3*dy<=7 &&
//@         get(c,r)==get(c+dx,r+dy)==get(c+2*dx,r+2*dy)==get(c+3*dx,r+3*dy) &&
//@         get(c,r)!=0)))))
public /*@ pure @*/ boolean winner() ;

/@ensures (\forall int c; c>=1 && c<=7; (\forall int r; r>=1 && r<=6; get(c,r)==0));
public void clear() ;
```

Domanda b)

Si specifichi in JML l'invariante pubblico del tipo `Connect4`.

Soluzione

```
//@ public invariant (\forall int c; c>=1 && c<=7; (\forall int r; r>=1 && r<=6;
//@       ( get(c,r)==0 || get(c,r)==1 || get(c,r)==2 ) &&
//@       ( get(c,r)!=0 => (\forall int r1; r1>=1 && r1<=r; get(c,r1)!=0))))
```

Domanda c)

Si consideri il tipo `Space4` che modella una variante del gioco `Connect4` seconda la quale il metodo `put` posiziona la pedina del giocatore indicato non "in cima" alla colonna ma in una posizione libera casuale (della colonna indicata).

Il tipo `Space4` può essere un sottotipo di `Connect4` secondo il principio di sostituibilità di Liskov? Motivare la propria risposta. Indicare poi (motivando la risposta) se sia possibile il contrario.

Soluzione

Il tipo `Space4` non può essere un sottotipo di `Connect4` in quanto ne violerebbe l'invariante pubblico che stabilisce che le colonne siano riempite dal basso (senza spazi vuoti). Verrebbe violata anche la postcondizione del metodo `put`.

Il contrario è invece possibile poichè l'ordine con il quale le pedine vengono fatte "cadere" nel gioco originale rappresenta un caso particolare dell'ordine arbitrario (non deterministico) previsto per il gioco `Space4`.

Esercizio 2

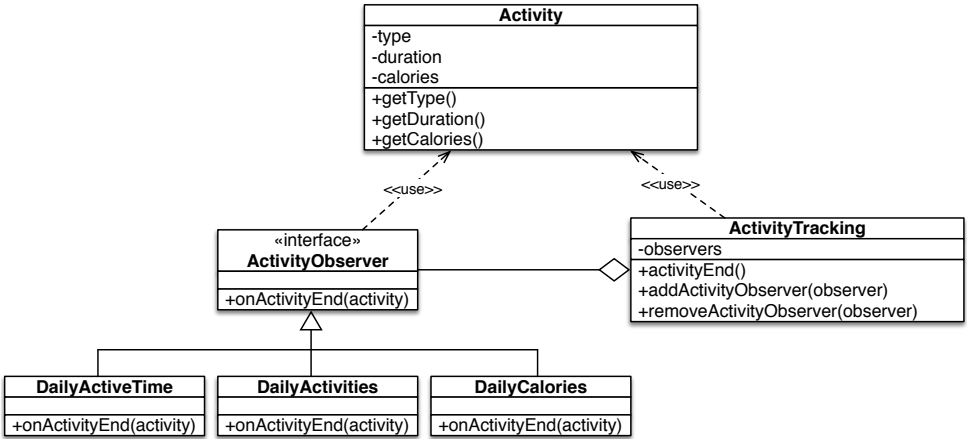
Un'app per smartwatch contiene un componente `ActivityTracking` che registra le attività dell'utente. Una attività (classe `Activity`) è caratterizzata da un tipo (nuoto, ciclismo, corsa, ...), una durata in minuti e una stima delle calorie bruciate. Il componente `ActivityTracking` contiene, tra gli altri, un metodo per terminare l'attività corrente. Quando un'attività termina, diversi altri componenti devono essere aggiornati: il componente `DailyCalories`, che salva il numero di calorie bruciate durante la giornata, il componente `DailyActivities`, che salva la lista dei tipi di attività completate durante la giornata, e il componente `DailyActiveTime`, che salva il numero totale di minuti di attività durante la giornata.

Si disegni una possibile architettura per questo sistema usando un diagramma delle classi UML e si fornisca una bozza di implementazione Java per le classi principali. Nello scrivere il codice Java si assuma che la classe `Activity` sia già implementata e si mostri l'implementazione della classe `ActivityTracker`, di una tra `DailyCalories`, `DailyActivities` e `DailyActiveTime`, e di ogni altra classe che possa essere rilevante.

Si favoriscano soluzioni che permettono future modifiche ed estensioni, considerando in particolare il caso in cui l'insieme dei componenti che devono essere aggiornati al termine di un'attività possa cambiare in futuro.

Soluzione

La soluzione consiste nell'adottare il pattern observer.



```
public class Activity {
  private final String type;
  private final int duration;
  private final int calories;

  public Activity(String type, int duration, int calories) {
    this.type = type;
    this.duration = duration;
    this.calories = calories;
  }

  public final String getType() { return type; }
  public final int getDuration() { return duration; }
  public final int getCalories() { return calories; }
}

public interface ActivityObserver {
  public void onActivityEnd(Activity activity);
}

public class ActivityTracking {
  private final List<ActivityObserver> observers = new ArrayList<>();

  public void addActivityObserver(ActivityObserver observer) {
    observers.add(observer);
  }

  public void removeActivityObserver(ActivityObserver observer) {
    observers.remove(observer);
  }

  public void activityEnd() {
    final Activity activity = new Activity(...);
    for (final ActivityObserver observer : observers) {
      observer.onActivityEnd(activity);
    }
  }
  ...
}

public class DailyActiveTime implements ActivityObserver {
  private int dailyActiveTime;

  @Override
  public void onActivityEnd(Activity activity) {
    dailyActiveTime += activity.getDuration();
  }
  ...
}

public class DailyActivities implements ActivityObserver {
  private final Set<String> dailyActivities = new HashSet<>();

  @Override
  public void onActivityEnd(Activity activity) {
    dailyActivities.add(activity.getType());
  }
  ...
}

public class DailyCalories implements ActivityObserver {
```

```
  private int dailyCalories;

  @Override
  public void onActivityEnd(Activity activity) {
    dailyCalories += activity.getCalories();
  }
  ...
}
```

Esercizio 3

In un'app per smartwatch, la classe `Stats` salva statistiche relative ad attività e salute dell'utente. In particolare, la classe contiene una lista `activities` con le attività completate e una lista `nutrients` con le sostanze nutritive assunte.

Domanda a)

Si implementino i seguenti metodi della classe `Stats`:

- `void addActivity(Activity activity)`: aggiunge `activity` alla lista `activities`;
- `void addNutrient(Nutrient nutrient)`: aggiunge `nutrient` alla lista `nutrients`;
- `List<Activity> getActivities()`: ritorna una copia della lista `activities`;
- `List<Nutrient> getNutrients()`: ritorna una copia della lista `nutrients`;

Si assuma che diversi thread possono invocare i metodi elencati sopra in maniera concorrente. Si favoriscano soluzioni che massimizzino le possibilità di esecuzione parallela: per esempio, due thread dovrebbero essere in grado di aggiungere una `activity` e un `nutrient` senza interferire tra loro.

Soluzione

Possiamo sincronizzare i metodi che accedono ad `activities` sull'oggetto `activities` e i metodi che accedono a `nutrients` sull'oggetto `nutrients`.

```
public class Stats {
  private final List<Activity> activities;
  private final List<Nutrient> nutrients;

  public Stats() {
    activities = new ArrayList<>();
    nutrients = new ArrayList<>();
  }

  public void addActivity(Activity activity) {
    synchronized (activities) { activities.add(activity); }
  }

  public void addNutrient(Nutrient nutrient) {
    synchronized (nutrients) { nutrients.add(nutrient); }
  }

  public List<Activity> getActivities() {
    final List<Activity> result = new ArrayList<>();
    synchronized (activities) { result.addAll(activities); }
    return result;
  }
```

```
  public List<Nutrient> getNutrients() {
    final List<Nutrient> result = new ArrayList<>();
    synchronized (nutrients) { result.addAll(nutrients); }
    return result;
  }
}
```

Domanda b)

Si assuma ora che si voglia anche salvare una variabile `caloriesBalance` che registra la somma delle calorie ottenute mediante sostanze nutritive meno la somma di tutte le calorie consumate durante attività. La variabile `caloriesBalance` viene aggiornata quando si aggiunge una sostanza nutritiva o un'attività, e può essere letta mediante il metodo `int getCaloriesBalance()`.

Si modifichi l'implementazione della classe `Stats` per soddisfare i nuovi requisiti. Si assuma che sia la classe `Nutrient` che la classe `Activity` forniscano un metodo `getCalories()` che fornisce le calorie fornite dalla sostanza nutritiva o consumate dall'attività.

Soluzione

Ora i metodi `addActivity()` e `addNutrient()` aggiornano una variabile condivisa. Di conseguenza, tutti i metodi devono essere sincronizzati sullo stesso oggetto (ad esempio `this`).

```
public class Stats {
  private final List<Activity> activities;
  private final List<Nutrient> nutrients;
  private int caloriesBalance;

  public Stats() {
    activities = new ArrayList<>();
    nutrients = new ArrayList<>();
    caloriesBalance = 0;
  }

  public synchronized void addActivity(Activity activity) {
    activities.add(activity);
    caloriesBalance -= activity.getCalories();
  }

  public synchronized void addNutrient(Nutrient nutrient) {
    nutrients.add(nutrient);
    caloriesBalance += nutrient.getCalories();
  }

  public synchronized List<Activity> getActivities() {
    return new ArrayList<>(activities);
  }

  public synchronized List<Nutrient> getNutrients() {
    return new ArrayList<>(nutrients);
  }

  public synchronized int getCaloriesBalance() {
    return caloriesBalance;
  }
}
```

Esercizio 4

Si consideri il seguente metodo statico, che ha la precondizione che ciascun elemento di `nums` é una stringa corrispondente a una rappresenatazione testuale di un numero intero.

```
public static List<Integer> addX(List<String> nums, int x) {
    List<Integer> plusX = new LinkedList<>();
    for(String numString : nums) {
        number = Integer.valueOf(numString);
        if (number>0)
          plusX.add(number + x);
    }
    return plusX;
}
```

Domanda a)

Si definisca un insieme minimale di test che copra tutte le condizioni e diramazioni (branches and conditions) del codice dato.

Soluzione

`nums = {"1", "-1"}, x=1`

Domanda b)

Quanti sono i cammini nel codice se il valore del parametro `nums` é una lista di 10 elementi che contiene esattamente 2 stringhe rappresentanti due interi positivi?

Soluzione

$9+8+7+\ldots+1 = \frac{9*10}{2} = 45$. (ci sono 9 possibili cammini quando l'elemento in posizione 0 é positivo e un elemento fra le posizioni da 1 a 9 é positivo; altri 8 cammini quando l'elemento in posizione 0 é positivo e un elemento fra le posizioni da 1 a 9 é positivo, ecc.)

Domanda c)

Si riscriva il metodo utilizzando i costrutti della programmazione funzionale di Java 8.

Soluzione

```
public static List<Integer> addX(List<String> nums, int x) {
    return nums.stream()
               .map(Integer::valueOf);
               .filter(number -> number>0)
               .map(number -> number + x)
               .collect(Collectors.toList());
}
```